深 圳 大 学 实 验 报 告

课程名称 Python 程序设计

项目名称 基于 NumPy 的 ViT-B/32 图像编码器复现

学　　院 人工智能学院

专　　业 人工智能 IEEE

指导教师 舒婷

报 告 人 裴曼秋　　**学号** 2025681082

实验时间 2026.11.9 - 2026.11.13

提交时间 2026.11.13

教务处制

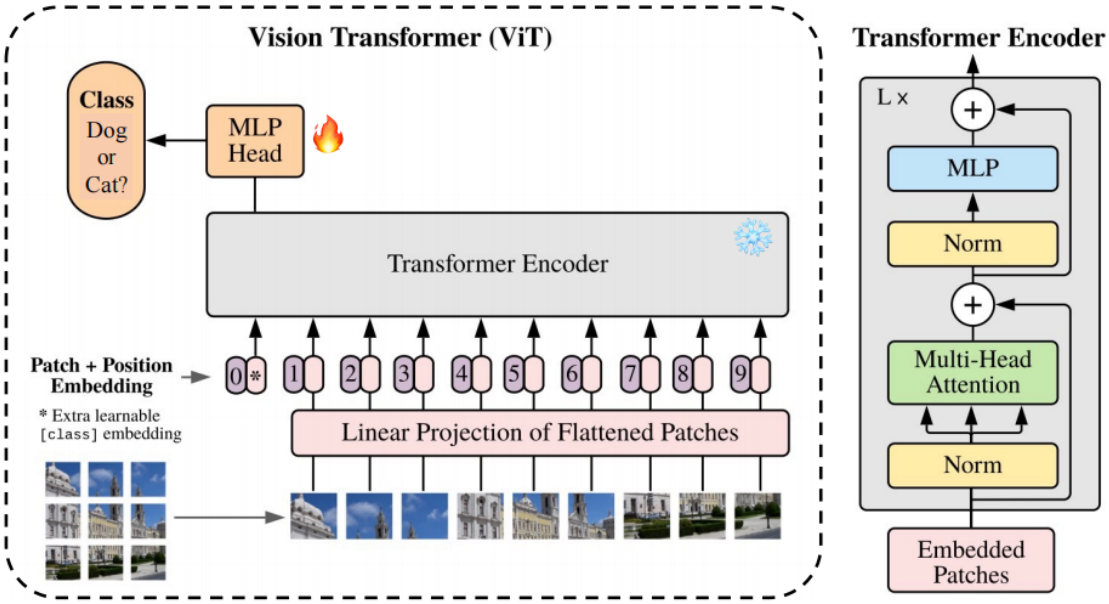


Figure 1: Vision Transformer 整体架构：patch 与位置编码输入 Transformer Encoder，最终由 CLS 向量输出特征表示

1 Objectives 实验目标

本实验在纯 NumPy 环境下完成，不安装 PyTorch、不联网下载模型、也不进行参数训练。依据课程提供的已训练权重，复现 OpenCLIP ViT-B/32 图像编码器的前向计算：输入一批经过预处理的图片，输出每张图一行 512 维、长度归一化的特征向量，并与独立的 FP32 参考结果在数值上对齐。完成后的编码器将作为后续图文匹配 CLIP 大作业的图像塔复用。此外，配套完成 12 道 NumPy 练习题，巩固数组形状、矩阵运算、汇总与广播等基础操作。

模型固定配置为：输入图像 224×224，patch 大小 32，共 7×7=49 个 patch；特征宽度 768，12 个注意力头，头维 64，12 层 Transformer Block，MLP 中间宽度 3072，最终输出维度 512。

2 Overview 方法概述

整体流程为“切 patch → 嵌入编码 → 逐层 Transformer 加工 → 取 CLS 输出”，如图 1 所示：

Spot 1 图像预处理：等比缩放后中心裁剪到 224×224，按通道均值与方差归一化，组织为 batch 数组 $[B, 3, 224, 224]$；

Spot 2 Patch 嵌入：切分为 7×7=49 个 32×32 方块并展平为 3072 维，线性投影到 768 维；前置拼接可学习的 CLS 向量，加入位置编码，得到 $[B, 50, 768]$；

Spot 3 Transformer 编码：经 pre-Norm 后通过 12 层 Block（多头注意力加 MLP，各带一条残差），取 CLS 行经后归一化、投影到 512 维并单位化，得到 $[B, 512]$。

3 Implementations 实现细节

本人贡献：在理解编码器接口与数据流的基础上，补全 `image_encoder.py` 中 TODO-02 至 TODO-06 的全部实现；完成 12 道 NumPy 练习题；整理 README 与本报告。课程提供的预处理、LayerNorm、精确 GELU、注意力 softmax 与单位化工具均未改动。

如图 1 所示，关键实现分为三部分：

Part 1 Patch 化与嵌入：`patchify` 将 $[B, 3, 224, 224]$ 重排为 7×7 patch 网格并按 C,H,W 顺序展平为 $[B, 49, 3072]$；patch 投影无 bias，使用 `patch_weight.T` 完成线性变换；前置拼接 CLS 后按行加入位置编码。

Part 2 多头注意力与 Block：`split_heads` 把 768 维特征 reshape 为 $[B, N, 12, 64]$ 再 transpose 为 $[B, 12, N, 64]$；QK 分数除以 $\sqrt{64}$，softmax 后加权 Value，再合并头并输出投影。Block 含两条残差：先加注意力输出，再经 Norm 2、GELU 激活的 MLP 后加第二次残差。

Part 3 输出头：选取第 0 行 CLS，经 post-Norm 后投影到 512 维；`visual.proj` 已是 [输入, 输出] 布局，不再转置，最后由单位化函数归一化。

权重方向约定：注意力与 MLP 线性权重按 [输出, 输入] 存储，矩阵乘时使用 `weight.T`；最终投影矩阵除外。关键代码如下：

```
# split_heads: [B,N,D] -> [B,H,N,D/H]
x = x.reshape(B, N, self.heads, self.head_dim)
x = x.transpose(0, 2, 1, 3)

# scores: q [B,H,N,64] @ k^T [B,H,64,N] -> [B,H,N,N]
scores = q @ k.swapaxes(-1, -2) / np.sqrt(np.float32(self.head_dim))
```

4 Results 结果

`check.py` 将实现结果与独立 FP32 参考逐阶段比对，检查形状、float32 类型、有限值，并按阶段使用绝对误差容忍（atol）与相对误差 rtol=2e-4。八个阶段全部通过，实测误差如表 1。

阶段	形状	最大绝对误差	平均绝对误差	容忍 atol
preprocess	(10, 3, 224, 224)	0.000e+00	0.000e+00	2e-6
patches	(2, 49, 3072)	0.000e+00	0.000e+00	2e-4
tokens	(2, 50, 768)	4.005e-05	1.289e-06	2e-4
block0	(2, 50, 768)	7.063e-06	3.250e-07	2e-4
last_tokens	(2, 50, 768)	4.768e-05	2.696e-06	2e-4
output	(10, 512)	5.439e-07	8.311e-08	2e-5
batch	(3, 512)	4.619e-07	7.632e-08	2e-5

Table 1: 分阶段数值比对误差

batch 阶段故意打乱图片顺序并重复输入图片，输出仍与参考一致，说明结果来自真实前向计算，而非写死参考行。`run.py` 对 10 张图输出形状 (10, 512) 的 float32 向量，每行长度均为 1。

5 Discussion 讨论

5.1 一个形状变化：注意力分头与合头

多头注意力需要把 768 维特征平均拆给 12 个头。`split_heads` 先 reshape 为 $[B, N, 12, 64]$，再 `transpose(0,2,1,3)` 得到 $[B, 12, N, 64]$；计算完成后按逆操作转回 $[B, N, 12, 64]$，再 reshape 合并为 $[B, N, 768]$。拆分与合并须保证每个头拿到连续的 64 维切片，且 token 顺序在前后保持不变。

5.2 一个修复的 Bug：transpose 不接受负索引

计算 QK 分数时最初写成 `k.transpose(-1,-2)` 只交换最后两轴，在 block0 阶段报错“axes don't match array”。原因是 NumPy 的 `transpose` 位置参数只接受非负轴序号，负索引不会被解析为“倒数第几个轴”。改用 `k.swapaxes(-1,-2)` 后问题解决：`swapaxes` 支持负轴，且只交换最后两轴，保留 batch 与头的分组，分数形状正确得到 $[B, H, N, N]$。

5.3 不足与改进

课程提供的 GELU 采用逐值循环调用 `math.erf`，在批量较大时速度有限；本作业图片规模较小，精度与速度均可接受，后续可向量化该激活函数以提升效率。

5.4 所用协助

实现参考了课程讲义与 `PPT-CODE-MAP.txt` 的代码—公式对照、`transformer_steps.py` 微型示范、`helpers.py` 数值工具，以及 NumPy 官方文档；报错定位与形状核对借助 AI 编程助手 (豆包）辅助排查，最终代码、校验结论与练习结果均由本人在本地运行确认。

指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

备注：

注： 1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。